Clasificación de modelos de carros mediante CNN's y propuestas de mejora

Yael Charles Marin
A01711111
Tecnológico de Monterrey Campus Querétaro
Inteligencia artificial avanzada para la ciencia de datos
Grupo 601

***Abstract*—La clasificación de modelos de carros exige distinguir características visuales semejantes bajo cambios de perspectiva, iluminación y fondo. Este trabajo reconstruye la evolución de dos CNN entrenadas desde cero y de EfficientNet-B0 con transferencia de aprendizaje sobre diez clases del dataset Car Model Image Classification. Se parte de 2,084 imágenes y de un análisis exploratorio de calidad, duplicados, balance y procedencia. La CNN v1 original, con entrada de 224×224 y 115,242 parámetros, obtiene una exactitud de prueba de 0.310 y F1 macro de 0.273. En CNN v2 se estudian dropout, selección por F1 macro, pooling, búsqueda de hiperparámetros, mayor resolución, augmentación y regularización L2. Una etapa con augmentación alcanza F1 macro de prueba de 0.325, pero la configuración posterior con L2 obtiene 0.274, evidenciando que mejorar validación no garantiza generalizar mejor. EfficientNet-B0 eleva el F1 macro de prueba desde 0.564 con la base congelada hasta 0.676 en la configuración final, que incluye ajuste fino, entrada de 320×320 y una revisión visual con 101 exclusiones. Esta última ejecución obtiene exactitud de prueba de 0.681. Se distinguen las mejoras observadas de sus posibles causas, pues también cambian las particiones y el control de la tasa de aprendizaje. Persisten sobreajuste, confusiones entre clases y posibles sesgos de procedencia. Los experimentos muestran la utilidad conjunta de representaciones preentrenadas, evaluación por clase y revisión de los datos, sin constituir una ablación controlada.**

***Palabras clave*—Clasificación de carros, CNN, EfficientNet-B0, transferencia de aprendizaje, regularización, augmentación, calidad de datos, F1 macro.**

I. Introducción

Identificar el modelo de un carro a partir de una imagen parece una tarea sencilla cuando se observan sus logos o su forma completa. Sin embargo, las fotografías pueden mostrar únicamente un faro, el interior o una vista parcial. También pueden contener varios vehículos, modificaciones estéticas y fondos que se repiten. Por esta razón, el problema no consiste solamente en identificar que hay un carro, sino en encontrar características que permitan distinguir una categoría de otra a pesar de esas variaciones.

Una red neuronal convolucional, o CNN, permite aprender patrones visuales mediante filtros que se ajustan durante el entrenamiento. En este proyecto se comparan una CNN propia inicial, su evolución CNN v2 y EfficientNet-B0 con pesos preentrenados. El interés está en analizar qué información recibe cada red, qué se conserva o se pierde al preparar las imágenes y hasta qué punto el desempeño observado se mantiene en ejemplos que no actualizan los pesos.

Se toma como referencia el análisis exploratorio de imágenes de Zhang et al. (2024), que propone pasar del procesamiento y las mediciones a la identificación e interpretación de patrones. Este enfoque resulta útil para relacionar los hallazgos del dataset con decisiones concretas, sin considerar que toda diferencia observada representa automáticamente un defecto. El estudio de Khan et al. (2026) aporta una discusión sobre desbalance y evaluación de clases minoritarias. Por su parte, Appasami y Savarimuthu (2025) estudian arquitecturas ligeras y el efecto de configurar convoluciones, pooling y entrenamiento.

En particular, Khan et al. (2026) comparan modificaciones de la función de pérdida, muestreo y aumento de datos. Sus resultados muestran que incrementar ejemplos no garantiza una mejora equivalente de las clases minoritarias. Esto respalda analizar la recuperación por categoría antes de elegir una estrategia de balance.

Appasami y Savarimuthu (2025) estudian conjuntamente kernel, pooling y tamaño de lote. Su configuración destacada utiliza kernel 4×4, pooling 4×4 y lote de 64, además de mecanismos como dropout y aumento de datos.

II. Fuente de datos y análisis exploratorio

A. Origen, tamaño y características

Se utiliza el dataset *Car Model Image Classification*, publicado en Kaggle por el usuario lightangle (lightangle, s.f.). Al momento de cargar los datos, se conserva una copia local de las imágenes y sus metadatos, lo que permite separar el material de origen de las transformaciones posteriores.

El dataset analizado contiene 2,084 imágenes distribuidas en diez clases. Los metadatos incluyen clase, nombre del archivo, título, creador, licencia, versión de licencia, fuente y enlaces de procedencia. La fuente declarada es Flickr en 2,083 registros y Sketchfab en uno.

La Tabla I muestra todas las clases que contiene el dataset y la distribución de imágenes que pertenecen a cada clase.

B. Balance de clases y diversidad visual

BMW M5 es la clase original más numerosa, con 239 imágenes, mientras que Audi RS5 tiene 124. En terminos de

TABLE I
DISTRIBUCIÓN DE LOS DATOS POR CLASE

Clase	Original
Audi R8	231
Audi RS5	124
BMW M3	233
BMW M5	239
Chevrolet Camaro	225
Ford Mustang	238
Mercedes AMG GT	147
Nissan GT-R	216
Porsche 911	225
Toyota Supra	206
Total	2,084

porcentaje son aproximadamente 11.47% y 5.95%, respectivamente, y la razón entre ambos conteos es 1.93. Hay un desbalance que debe tomarse en cuenta.

Una posible consecuencia de este desbalance es que acertar en las clases más frecuentes puede elevar la exactitud global aun cuando otras categorías se reconozcan poco. Por ello, es importante considerar esto más adelante.

La Figura 1 muestra cuatro ejemplos por clase, seleccionados de forma aleatoria. Se observan diferentes cambios en la perspectiva, iluminación y fondo. También aparecen interiores, acercamientos y escenas con varios vehículos. Estos ejemplos sugieren que el dataset incluye tanto variación visual válida como posibles problemas de relevancia o etiquetado.

C. Dimensiones, proporciones y calidad visual

La Figura 2 resume dimensiones, resolución, relación de aspecto y tamaño del archivo. El ancho mediano es de 1,024 píxeles y el alto mediano es de 685, estos valores representan medianas marginales y no implican que todas las imágenes tengan esa combinación, la resolución mediana es 0.701 megapíxeles, con valores entre 0.154 y 2.074, finalmente, la relación ancho/alto varía entre 0.562 y 3.373. Por lo tanto, forzar directamente todas las imágenes a un cuadrado modificaría sus proporciones.

El tamaño mediano de una imagen es 186.362 KiB, es importante distinguir que un archivo mayor no necesariamente aporta más información útil, además el modelo requiere una entrada uniforme que conserve la geometría de la carrocería.

En la Figura 3 se observan distribuciones y valores extremos por clase.

La nitidez aproximada tiene una mediana de 2,150.608 y una cola hacia la derecha (valores altos), por ejemplo un fondo con muchos bordes puede aumentar este indicador sin que el carro se distinga mejor, un brillo extremo no justifica eliminar una imagen.

D. Procedencia y concentración de autores

En BMW M5, 218 de las 239 imágenes originales, equivalentes a 91.21%, pertenecen al creador nakhon100. Ese mismo creador aporta 116 de las 233 imágenes de BMW M3, o 49.79%. En Audi R8, Prior Design NA concentra 135 de 231 imágenes, aproximadamente 58.44%. La Figura 4 muestra estas concentraciones.

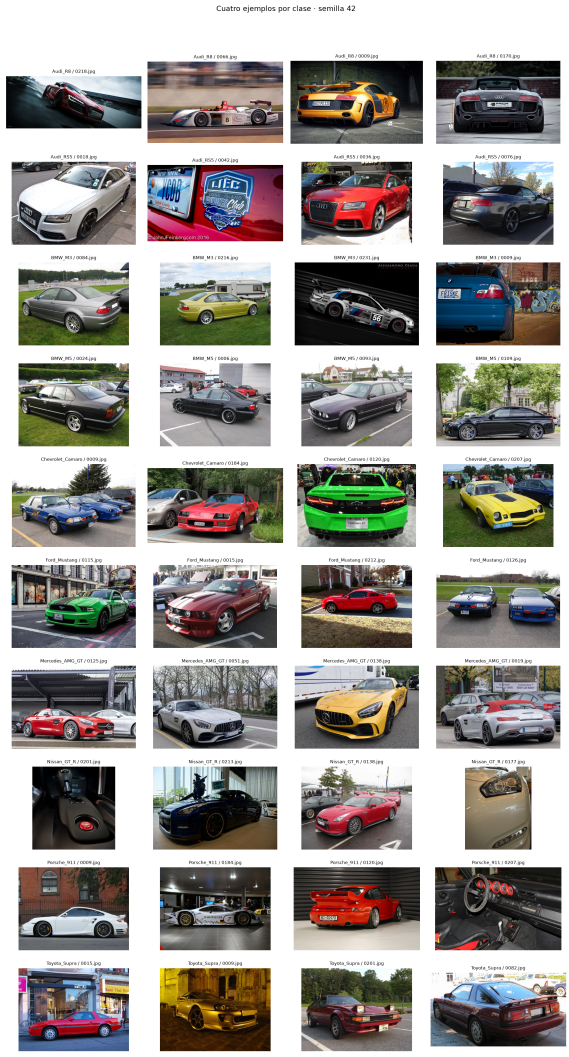

Fig. 1. Cuatro ejemplos originales por clase, con semilla 42.

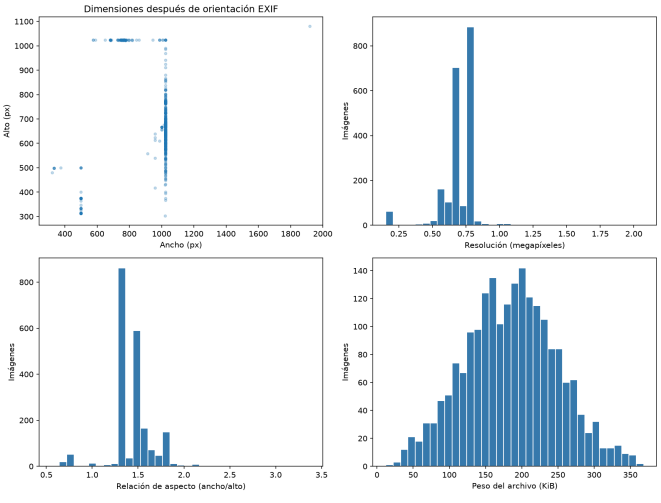

Fig. 2. Dimensiones corregidas por orientación EXIF, resolución, relación de aspecto y peso de las imágenes originales.

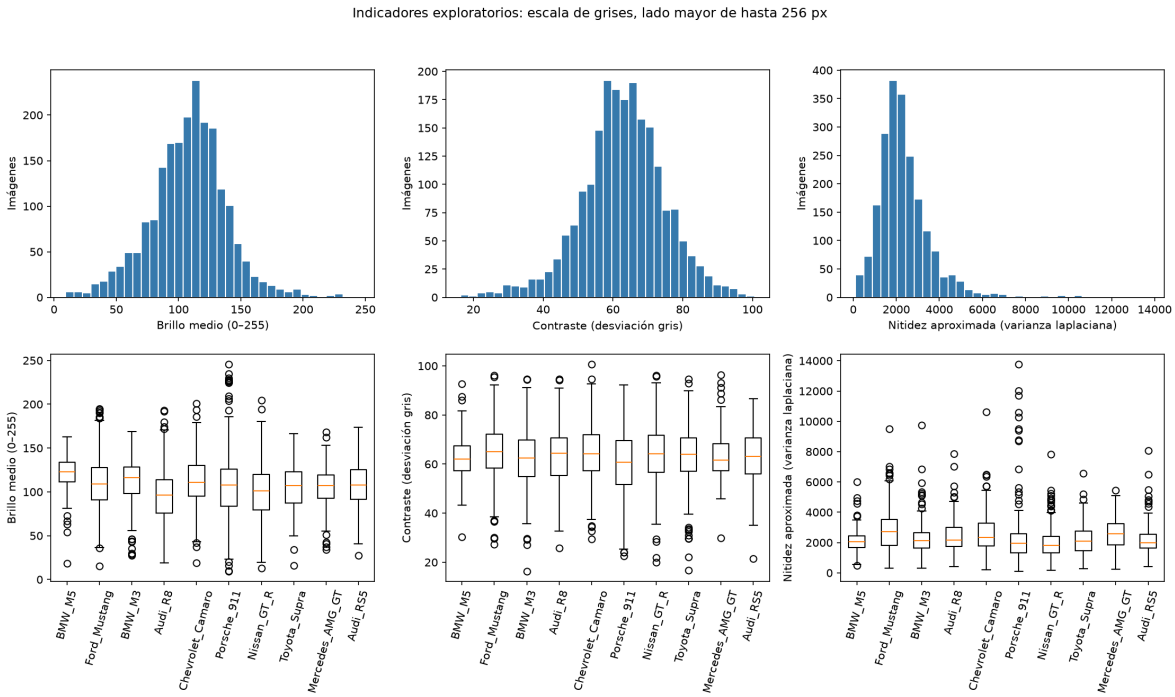

Fig. 3. Indicadores exploratorios de brillo, contraste y nitidez aproximada, globales y por clase.

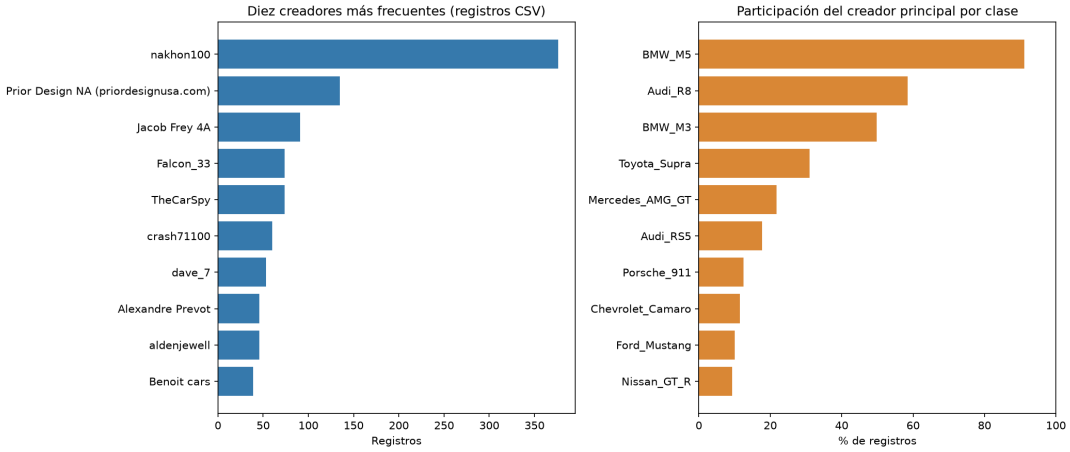

Fig. 4. Creadores más frecuentes y participación del principal creador en cada clase original. La concentración es particularmente alta en BMW M5. Fuente: análisis del campo de creador en los metadatos.

Esto es importante porque si un autor utiliza escenarios, encuadres o marcas de agua similares, la red podría asociarlos con una clase. Esta posibilidad limita la interpretación de una partición aleatoria por imagen, ya que muchas fotografías no necesariamente representan una diversidad equivalente de vehículos o condiciones de captura. El aprendizaje de estos atajos permanece como una hipótesis.

III. Limpieza, transformación y conjuntos de datos

Esta sección y las dos siguientes describen la ejecución original de CNN v1: entrada de 224×224, suavizado gaussiano, pooling con paso cuatro y selección por mínima pérdida de validación. Se conserva esta referencia histórica para interpretar sus reportes. El código actual utiliza otras dimensiones y criterios, que se documentan en las etapas posteriores; volver a ejecutarlo no reproduce automáticamente esta configuración inicial.

A. Duplicados y conflictos de etiquetas

Se comparan dos formas de identidad exacta, SHA-256 de los archivos e identidad de los píxeles RGB después de corregir la orientación, la primera identifica archivos iguales, mientras que la segunda permite reconocer contenido decod-

ificado idéntico. En este dataset, ambos criterios señalan los mismos nueve pares, es decir, 18 rutas únicas.

Seis pares tienen etiquetas distintas pero representan la misma imagen o muy similar: uno entre BMW M3 y BMW M5, y cinco entre Chevrolet Camaro y Ford Mustang. La Figura 5 presenta algunos ejemplos. En escenas con varios vehículos puede haber más de una categoría visible, pero asignar etiquetas diferentes a la misma imagen genera una supervisión ambigua para una tarea de una sola salida. Se excluyen ambos miembros de esos seis pares, sin intentar averiguar cuál etiqueta es correcta. De los tres pares restantes se conserva una copia por orden alfabético de ruta.

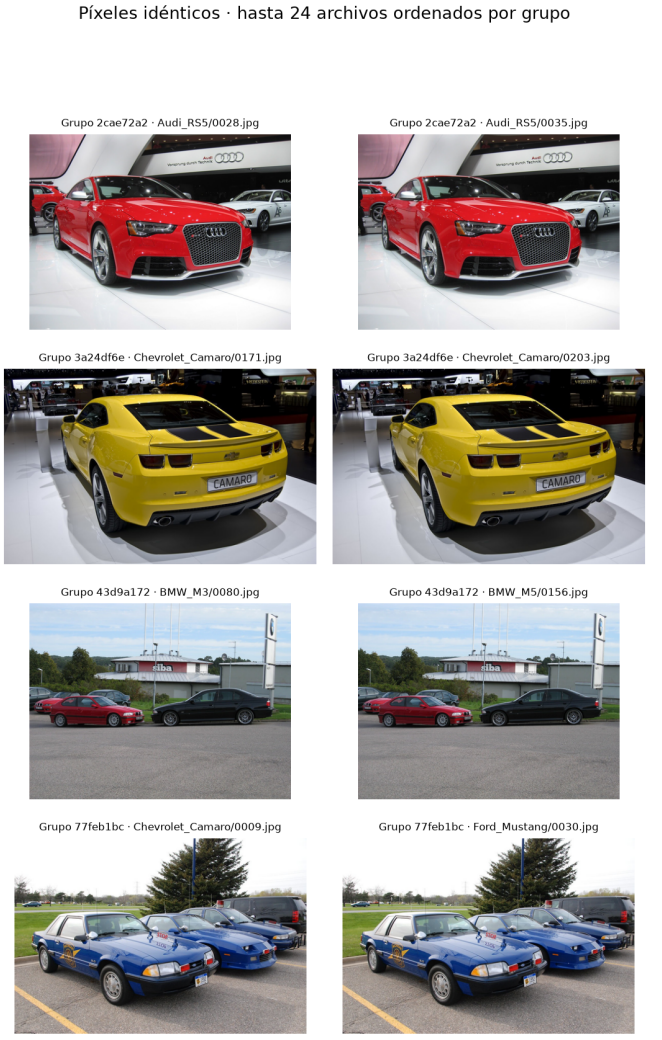

Fig. 5. Detalle de los primeros cuatro pares del mosaico de duplicados exactos, dos con la misma clase y dos con etiquetas distintas.

Además, se comparan huellas perceptuales dHash de 64 bits con distancia de Hamming de hasta cinco. Este criterio identifica tres pares candidatos dentro de Audi R8, Porsche 911 y Ford Mustang, con la Figura 6 muestra que el contenido incluye tanto fotografías como esquemas y material publicitario.

Una distancia pequeña en dHash indica una estructura visual parecida, sin confirmar duplicación. La exclusión automática puede reducir repetición o retirar variaciones útiles.

TABLE II
BALANCE DE IMÁGENES DURANTE LA LIMPIEZA

Concepto	Imágenes
Entrada original	2,084
Retiradas por conflicto de etiquetas	12
Copias exactas adicionales retiradas	3
Candidatos similares retirados	3
Dataset resultante	2,066

Nota. Las exclusiones son mutuamente excluyentes. El balance se obtiene de los reportes de duplicados y candidatos y de la selección implementada: $2{,}084-12-3-3 = 2{,}066$.

La Tabla II resume la reducción de imagenes, es importante mencionar que la limpieza ocurre antes de la división de conjuntos para evitar que las copias detectadas terminen en conjuntos distintos, lo que reduce una fuga de información.

B. Transformación de las imágenes

Primero se aplica la orientación EXIF y se convierte la imagen a RGB, de manera que la entrada tenga tres canales. Después se ajusta dentro de un cuadrado de 224×224, manteniendo la relación de aspecto y sin recortar el contenido. Se aplica un filtro gaussiano de radio 1 a la imagen ajustada y se completa el espacio restante con relleno negro centrado. El resultado se guarda como JPEG con calidad 95.

El ajuste proporcional evita deformar elementos como faros, parrillas y también evita recortes automáticos que podrían eliminar el carro, aunque mantiene fondos y objetos ajenos a la clase. El relleno introduce pixeles negros cuyo tamaño depende de la proporción original. Este tamaño elegido reduce el costo de trabajar con los originales, a cambio de perder pequeños detalles.

El suavizado gaussiano puede disminuir variaciones de alta frecuencia, pero también difuminar emblemas o bordes discriminantes.

Al cargar la imagen procesada se convierten sus intensidades a punto flotante de 32 bits y se normalizan mediante

$$\widetilde{x}_{i,j,c} = \frac{x_{i,j,c}}{255}, \qquad 0 \le \widetilde{x}_{i,j,c} \le 1, \tag{1}$$

donde $x_{i,j,c}$ es la intensidad del píxel en fila i, columna j y canal c. Esta transformación unifica la escala de entrada, lo que resulta en que la forma final de cada ejemplo es $224 \times 224 \times 3$.

C. Partición y carga para el aprendizaje

El conjunto procesado se divide en 70% para entrenamiento, 15% para validación y 15% para prueba, mediante dos separaciones estratificadas con semilla 42. La primera reserva 30% y la segunda divide esa reserva en partes iguales. El

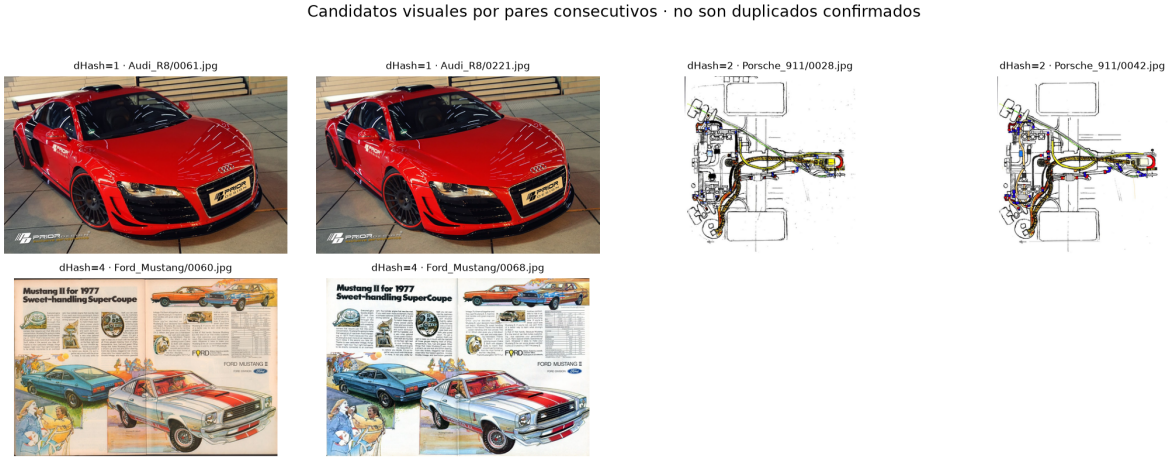

Fig. 6. Los tres pares candidatos por similitud dHash, ordenados por distancia.

redondeo produce 1,446 imágenes de entrenamiento y 310 en cada conjunto restante.

El conjunto de entrenamiento se utiliza para actualizar los pesos, validación permite observar el comportamiento fuera de esos ejemplos y elegir cuándo detenerse y prueba evalúa el modelo resultante. La estratificación mantiene aproximadamente las proporciones de clase y evita que las categorías menos frecuentes desaparezcan de algún conjunto.

Las imágenes se cargan por lotes de hasta 32 y las diez etiquetas se representan con enteros de 0 a 9 en orden alfabético, que representan únicamente la salida correspondiente.

IV. Conceptos y decisiones del modelo

A. *Convoluciones y aprendizaje de características*

La CNN busca aprender características útiles para separar las diez clases sin definir manualmente una variable para cada faro, rueda o parrilla.

Los pesos permiten buscar un patrón en distintas regiones con un mismo filtro, al aumentar los filtros se pueden representar más patrones y las capas posteriores combinan sus respuestas. Esto permite construir representaciones más complejas.

Las convoluciones implementadas tienen paso uno y relleno *same*, por lo que conservan las dimensiones espaciales de su entrada antes del pooling. Su cantidad de parámetros se calcula como

$$P_{\text{conv}} = (3 \cdot 3 \cdot C + 1)F, \tag{2}$$

con F filtros y un sesgo por filtro. La Tabla III presenta el recorrido completo. Por ejemplo, la primera convolución tiene $(9 \cdot 3 + 1)16 = 448$ parámetros.

B. *Función ReLU*

Después de cada convolución y en la capa densa intermedia se utiliza la unidad lineal rectificada:

$$\text{ReLU}(z) = \max(0, z). \tag{3}$$

TABLE III
Arquitectura de la CNN y parámetros entrenables

Operación	Salida	Parámetros
Entrada RGB	$224 \times 224 \times 3$	0
Conv. 3×3, 16, ReLU	$224 \times 224 \times 16$	448
Max pooling 4×4	$56 \times 56 \times 16$	0
Conv. 3×3, 32, ReLU	$56 \times 56 \times 32$	4,640
Max pooling 4×4	$14 \times 14 \times 32$	0
Conv. 3×3, 64, ReLU	$14 \times 14 \times 64$	18,496
Max pooling 4×4	$4 \times 4 \times 64$	0
Conv. 3×3, 128, ReLU	$4 \times 4 \times 128$	73,856
Max pooling 4×4	$1 \times 1 \times 128$	0
Aplanado	128	0
Densa, ReLU	128	16,512
Densa, softmax	10	1,290
Total		115,242

ReLU introduce una transformación no lineal que conserva los valores positivos y convierte los negativos en cero. En esta red, ReLU permite combinar las respuestas de los filtros de una forma que depende de los patrones presentes en cada imagen.

C. *Max pooling y reducción espacial*

Después de cada convolución se aplica max pooling con ventana y paso definidas. Esta técnica permite conservar la respuesta máxima de cada región, reduciendo las posiciones que procesan las capas siguientes sin añadir parámetros entrenables. El pooling conserva respuestas fuertes, aunque descarta la ubicación exacta de otras activaciones.

Con relleno *same*, las dimensiones de salida se redondean hacia arriba al dividir por cuatro. Por ello, la secuencia es $224 \to 56 \to 14 \to 4 \to 1$, de acuerdo con la definición de la operación (Keras, s.f.-b).

Esta reducción mantiene pequeño el número de conexiones densas, pero podría descartar detalles o relaciones espaciales necesarios. Los resultados de arquitecturas ligeras en otros dominios (Appasami & Savarimuthu, 2025) no prueban que esta reducción sea óptima para carros.

D. Capas densas, softmax y función de pérdida

El aplanado convierte la representación $1 \times 1 \times 128$ en 128 valores sin aprender nuevos pesos. La capa densa siguiente combina esos valores en 128 unidades ReLU. Finalmente, diez*logits*, se transforman con softmax:

$$\widehat{p}_{i,k} = \frac{\exp(z_{i,k})}{\sum_{j=1}^{K} \exp(z_{i,j})}, \qquad K = 10. \tag{4}$$

Aquí $z_{i,k}$ es la puntuación de la clase k para la imagen i, las salidas de todas las clases son positivas y deben sumar uno (100%), para seleccionar la predicción se usa la clase con mayor valor.

Se minimiza la entropía cruzada categórica con etiquetas enteras:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log \widehat{p}_{i,y_i}, \tag{5}$$

donde y_i es la clase correcta y N el número de ejemplos considerado. La denominación *sparse* indica que la etiqueta se proporciona como un entero, sin exigir una representación one-hot explícita.

La pérdida considera cuánta probabilidad se asigna a la clase correcta, no únicamente si esta obtiene el máximo, por lo que, un error muy grande puede aumentar mucho la pérdida aunque el número total de aciertos apenas cambie. Esta distinción permite interpretar las curvas cuando la exactitud se mantiene estable y la pérdida de validación aumenta.

V. Entrenamiento y evaluación de la CNN v1

A. Configuración e hiperparámetros

La red se entrena desde cero en CPU con TensorFlow/Keras, en cada lote se calcula la pérdida y se propaga su gradiente desde la salida hacia los parámetros de las capas anteriores mediante la regla de la cadena. Este proceso, conocido como retropropagación, determina cómo modificar los pesos para reducir el error observado.

Se utiliza Adam con tasa de aprendizaje 0.001, ya que, adam mantiene estimaciones de primer y segundo momento del gradiente para adaptar las actualizaciones de cada parámetro (Kingma & Ba, 2015) y la tasa controla la escala de esas actualizaciones, pero su valor no representa una garantía de convergencia ni de generalización. Por otro lado, Los lotes de 32 permiten actualizar la red con grupos de ejemplos. Con 1,446 imágenes se requieren 46 lotes por época.

B. Detención temprana y selección de pesos

El *early stopping* monitorea la pérdida de validación y detiene el entrenamiento después de cinco épocas consecutivas sin disminuirla. Al finalizar, restaura los pesos asociados con el menor valor registrado (Keras, s.f.-a). De esta forma se evita conservar automáticamente la última época, que puede ajustarse mejor al entrenamiento y peor a ejemplos reservados.

Las gráficas disponibles contienen 17 épocas, donde el mínimo de validación se aprecia alrededor de la época 12 y después la pérdida aumenta durante las cinco siguientes. La detención temprana limita la degradación posterior, pero no corrige por sí misma la falta de diversidad, las etiquetas ambiguas ni la información descartada durante el procesamiento.

TABLE IV
Configuración del entrenamiento analizado

Elemento	Configuración
Entrada	$224 \times 224 \times 3$, valores entre 0 y 1
Optimizador	Adam, tasa de aprendizaje 0.001
Pérdida	Entropía cruzada categórica con etiquetas enteras
Lote	Hasta 32 imágenes
Máximo de épocas	30
Detención temprana	Mínima pérdida de validación; paciencia 5
Pesos finales	Restauración de los mejores de validación
Semilla	42 para partición, inicialización y mezcla

En las curvas, entrenamiento es el promedio de los lotes mientras los pesos cambian durante la época. Validación y prueba se evalúan al terminarla. Las métricas finales se vuelven a calcular para los tres conjuntos con los pesos restaurados y sin mezclar el orden de evaluación.

C. Métricas utilizadas

La exactitud mide la proporción de imágenes cuya predicción coincide con la etiqueta:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}(\widehat{y}_i = y_i). \tag{6}$$

El indicador $\mathbf{1}$ vale uno cuando hay coincidencia y cero en caso contrario, es decir, para cada clase k, se consideran positivos sus ejemplos y negativos los de las otras nueve. Los verdaderos positivos TP_k son sus aciertos, los falsos positivos FP_k son imágenes ajenas asignadas a ella, y los falsos negativos FN_k son sus imágenes asignadas a otra categoría. Entonces,

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \tag{7}$$

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \tag{8}$$

$$F1_k = \frac{2\,\text{Precision}_k\,\text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}. \tag{9}$$

La precisión responde qué proporción de las predicciones de una clase es correcta. Recall indica qué proporción de sus ejemplos reales se recupera. F1 combina ambas mediante una media armónica y penaliza que alguna sea baja.

Para una métrica por clase m_k, el promedio macro se define como

$$m_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} m_k. \tag{10}$$

En particular, el F1 macro promedia los diez valores de $F1_k$, así se conserva la importancia de Audi RS5 y Mercedes AMG GT aunque tengan menos ejemplos.

VI. Resultados de la CNN v1 original

A. Desempeño global con los pesos restaurados

La Tabla V transcribe las etiquetas numéricas del reporte final mostrado en la Figura 7, que representan valores redondeados a tres decimales.

TABLE V
MÉTRICAS FINALES CON LOS MEJORES PESOS DE VALIDACIÓN

Conjunto	Accuracy	Prec. macro	Recall macro	F1 macro
Train	0.464	0.486	0.454	0.423
Validación	0.294	0.290	0.283	0.245
Prueba	0.310	0.304	0.297	0.273

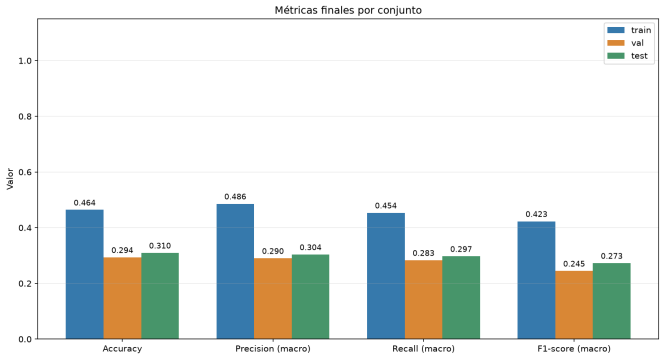

Fig. 7. Reporte de exactitud, precisión macro, recall macro y F1 macro, con los mejores resultados tras aplicar early stopping.

La exactitud de prueba indica que se reconoce aproximadamente 31% de los ejemplos reservados, mientras que cerca de 69% se asigna a otra clase. En la matriz se cuentan 96 aciertos de 310.

El F1 macro de prueba, 0.273, muestra que el equilibrio entre precisión y recuperación es limitado al dar el mismo peso a todas las categorías.

La diferencia aproximada entre exactitud final de entrenamiento y prueba es de 15.4 puntos porcentuales, y respecto de validación es de 17.0. El modelo aprovecha mejor los ejemplos usados para ajustar sus parámetros. Validación y prueba tienen resultados cercanos, aunque el valor algo mayor de prueba no demuestra una mejora,

B. Curvas de aprendizaje y sobreajuste

La Figura 8 permite observar una separación progresiva. La exactitud de entrenamiento aumenta hasta aproximadamente 0.62 en la época 17; validación y prueba varian alrededor de 0.2 a 0.3. Al mismo tiempo, la pérdida de entrenamiento disminuye, mientras que las pérdidas de evaluación dejan de mejorar y aumentan al final.

Este patrón es evidencia de sobreajuste, donde reducir el error en los ejemplos de aprendizaje ya no produce una mejora comparable fuera de ellos.

También se observa por qué la pérdida aporta información adicional a la exactitud, ya que la red puede conservar un número parecido de aciertos y volverse más confiada en sus errores. En este caso, la entropía cruzada aumenta aunque apenas cambie la clase con mayor probabilidad, por eso se utiliza la pérdida de validación para detenerse y no únicamente la exactitud.

C. Matriz de confusión y diferencias entre clases

La Figura 9 presenta cantidades absolutas, cada fila corresponde a una clase real y cada columna predicha, la diagonal contiene los aciertos. La Tabla VI divide los aciertos por el total real de cada fila.

TABLE VI
ACIERTOS Y RECUPERACIÓN POR CLASE EN PRUEBA

Clase	Aciertos	Soporte	Recall (%)
Audi R8	15	35	42.9
Audi RS5	1	18	5.6
BMW M3	1	35	2.9
BMW M5	26	35	74.3
Chevrolet Camaro	13	33	39.4
Ford Mustang	11	35	31.4
Mercedes AMG GT	8	22	36.4
Nissan GT-R	7	33	21.2
Porsche 911	9	33	27.3
Toyota Supra	5	31	16.1
Total	96	310	—

Nota. Soporte es el número real de ejemplos de la clase. La exactitud global es $96/310$, el recall macro promedia las diez tasas.

BMW M5 alcanza 26 aciertos de 35, mientras BMW M3 obtiene uno de 35, además, 16 imágenes de BMW M3 se predicen como BMW M5. Esta confusión es asimétrica porque solo una imagen de BMW M5 se asigna a BMW M3.

Audi RS5 también tiene una recuperación baja, debido a que únicamente uno de sus 18 ejemplos se identifica correctamente, una posible causa es su menor cantidad de entrenamiento, pero no explica por sí sola el resultado, ya que BMW M3 tiene muchos más ejemplos y presenta una recuperación todavía menor.

Otros resultados son que diez imágenes de Nissan GT-R y diez de Porsche 911 se asignan a Audi R8, a su vez, ocho Audi R8 y ocho Ford Mustang se predicen como Mercedes AMG GT. Estas cantidades muestran que el error no está restringido a parejas de modelos cercanos. La revisión de ejemplos concretos permitiría estudiar si predominan encuadres parciales, fondos compartidos o características del vehículo.

BMW M5 recupera buena parte de sus ejemplos, pero también recibe errores de otras filas y su recall alto no implica que todas sus predicciones sean correctas.

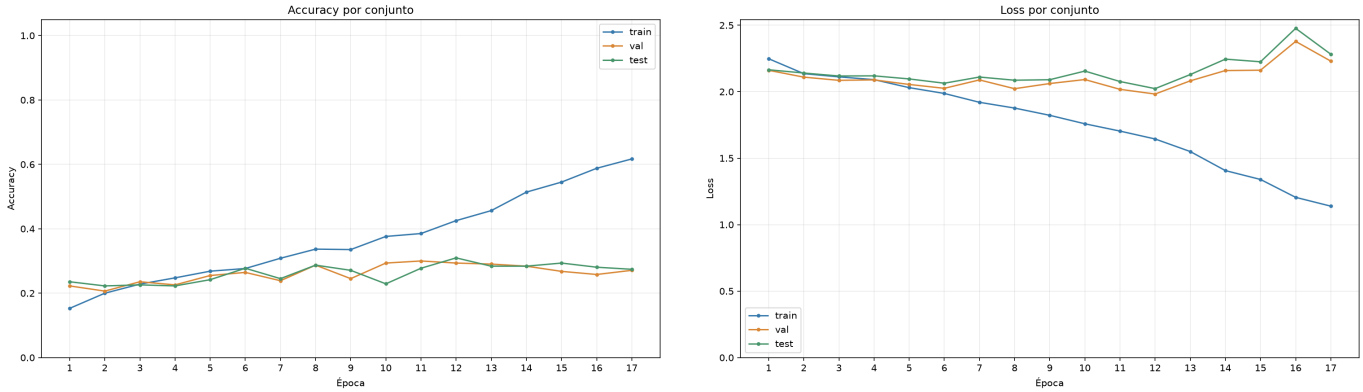

Fig. 8. Exactitud y pérdida durante las 17 épocas registradas.

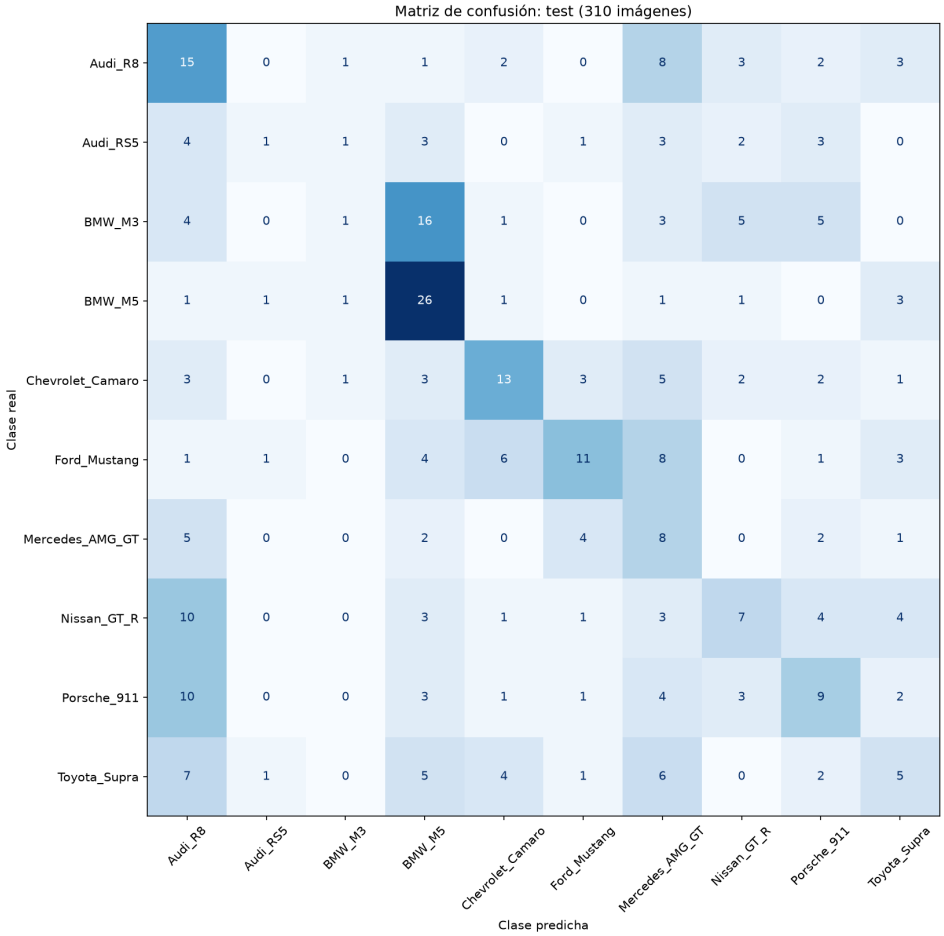

Fig. 9. Matriz de confusión final de prueba, con 310 imágenes. Las filas son clases reales y las columnas clases predichas.

VII. Evolución de la CNN v2

A. Mejoras aplicadas

Dropout desactiva aleatoriamente una fracción de las activaciones durante el entrenamiento, para reducir la dependencia de combinaciones particulares de unidades, por ello, la exactitud de entrenamiento registrada durante una época con dropout no es directamente equivalente a evaluar después esos mismos datos. Appasami y Savarimuthu (2025) incorporan dropout dentro de su diseño ligero para limitar el sobreajuste, aquí se adapta esa idea a la capa densa de la CNN de carros.

La *búsqueda de hiperparámetros* compara decisiones fijadas antes del entrenamiento, como la tasa de aprendizaje, la ventana del pooling o la probabilidad de dropout. En una búsqueda de malla se evalúan las combinaciones de los valores. Wojciuk

et al. (2024) muestran que la utilidad de optimizar esos valores depende de la tarea y del conjunto, y que la búsqueda puede ser costosa. Estos trabajos justifican comparar configuraciones mediante validación, sin asumir que un valor recomendado en otro dominio sea óptimo para carros.

La *regularización L2* añade un costo a la magnitud de los pesos, donde se busca limitar soluciones que ajusten demasiado los ejemplos de entrenamiento. Baressi Šegota et al. (2025) utilizan L2 en VGG16 para imágenes de linfoma con un valor de coeficiente de 0.01, el cual no se adopta directamente por diferencias en la arquitectura, el tamaño de los datos y el dominio.

La *augmentación* genera variaciones de los ejemplos conservando su etiqueta esperada. En fotografías de carros, un reflejo horizontal o una rotación pequeña puede representar otro encuadre del mismo modelo, sin embargo, añadir variantes no equivale a obtener nuevos vehículos independientes ni corrige una etiqueta incorrecta. El trabajo de Baressi Šegota et al. (2025) combina recortes en parches con transformaciones geométricas en histopatología, en este caso solo se adapta la idea de variar las imágenes, sin fragmentar los carros en parches ni introducir giros de 90 grados.

Finalmente, la *resolución* determina cuánto detalle entra a la red y el *stride* del pooling determina cuántas posiciones se conservan entre bloques. La ventana define la región sobre la que se toma el máximo, el stride define cuánto avanza, para este caso se definieron valores de padding *same* y stride 2

B. Experimentación en la ccn v2

La primera modificación sobre CNN v1 fue añadir dropout después de la capa densa de 128 unidades, se probaron tasas de 0.3 y 0.5 y se cambió el criterio de selección desde la pérdida de validación hacia el F1 macro de validación, la razón para cambiar esta metrica fue dar igual importancia a las diez clases, frente a los bajos recalls observados en Audi RS5 y BMW M3.

El uso de GPU permitió explorar una representación espacial mayor. Con entrada de 224×224 y stride 2, los tamaños pasan por 112, 56, 28 y 14, en lugar de terminar en una sola posición. Así, `Flatten` contiene $14 \cdot 14 \cdot 128 = 25{,}088$ valores y la red tiene 3,310,122 parámetros, frente a 115,242 en la referencia original. Igualmente, se compararon ventanas de pooling y posteriormente se revisó el efecto individual que tenían los hiperparametros de dropout, learning rate y pooling en la red, aumentando la capacidad de ajuste.

Las gráficas de búsqueda conservadas resumen esta exploración de hiperparametros, en ella se detectó que la combinación que produjo meores resultados fue learning rate 0.001, dropout 0.5 y ventana 3×3, con F1 macro de validación de 0.3574. Esta configuración no significa que es la mejor para todos los casos, sino que para este problema mostró el mejor rendimiento entre los diferentes valores evaluados.

Después se aumentó la entrada a 320×320, se retiró el filtro gaussiano y se incorporó augmentación al conjunto de train. Los cuatro bloques conservan ahora mapas de 160, 80, 40 y 20 posiciones por eje. `Flatten` contiene 51,200 valores y la red alcanza 6,652,458 parámetros, lo que se buscó con esta mejora fue una combinación de mayor detalle y mayor capacidad con una estrategia para ampliar las variaciones observadas.

Al aplicar augmentación se guarda una variante por imagen original del conjunto de train, donde se aplicó un reflejo horizontal con probabilidad 0.5, giro de hasta $\pm 5°$, desplazamiento de hasta 3% por eje, alejamiento de hasta 10% y gamma entre 0.9 y 1.1, estas variantes se generan después de dividir los conjuntos originales para que una transformación de una imagen de entrenamiento no termine en otro conjunto. Se mantienen los conjuntos de validación y prueba sin variantes. Tras aplicar este cambio, el conjunto de train pasa de 1,446 a 2,892 entradas.

Finalmente se revisaron diferentes valores de L2 con los valores de la Tabla VII, se encontró que la mejor configuración que producia un valor de F1 macro de validación correspondió a 10^{-4}, por lo que se quedó en la configuración posterior.

TABLE VII
COMPARACIÓN DE L2 CON ENTRADA DE 320×320

Coeficiente λ	Mejor F1 macro de validación
0	0.3264
10^{-5}	0.3286
10^{-4}	0.3357
10^{-3}	0.3048

Nota. Los demás hiperparametros fueron: Adam con learning rate 0.001, dropout 0.5, pooling 3×3, stride 2 y 6,652,458 parámetros en todos los candidatos.

C. Resultados y límites de las mejoras

La Tabla VIII reúne etapas de la evolución. El aumento de capacidad con stride 2 mejora el ajuste de train, pero el F1 de prueba baja a 0.252. La búsqueda conjunta y la inicialización controlada alcanzan 0.308 en prueba. La etapa con augmentación obtiene 0.325, mientras que el reporte posterior con L2 regresa a 0.274. La selección se hizo por validación; estos resultados de prueba se usan para describir retrospectivamente lo ocurrido, no para redefinir al ganador de la búsqueda.

TABLE VIII
EVOLUCIÓN HISTÓRICA DE LAS CNN; MÉTRICAS REDONDEADAS

Etapa	Acc. prueba	F1 val.	F1 prueba
CNN v1 original	0.310	0.245	0.273
CNN v2, stride 2	0.274	0.339	0.252
Búsqueda e inicialización controlada	0.319	0.357	0.308
Augmentación, 320 y sin suavizado	0.348	0.321	0.325
Configuración posterior con L2	0.297	0.336	0.274

Nota. F1 corresponde al promedio macro. Cada ejecución utiliza 310 imágenes de prueba. Los cambios entre etapas no constituyen una ablación controlada; train incluye variantes en las dos últimas.

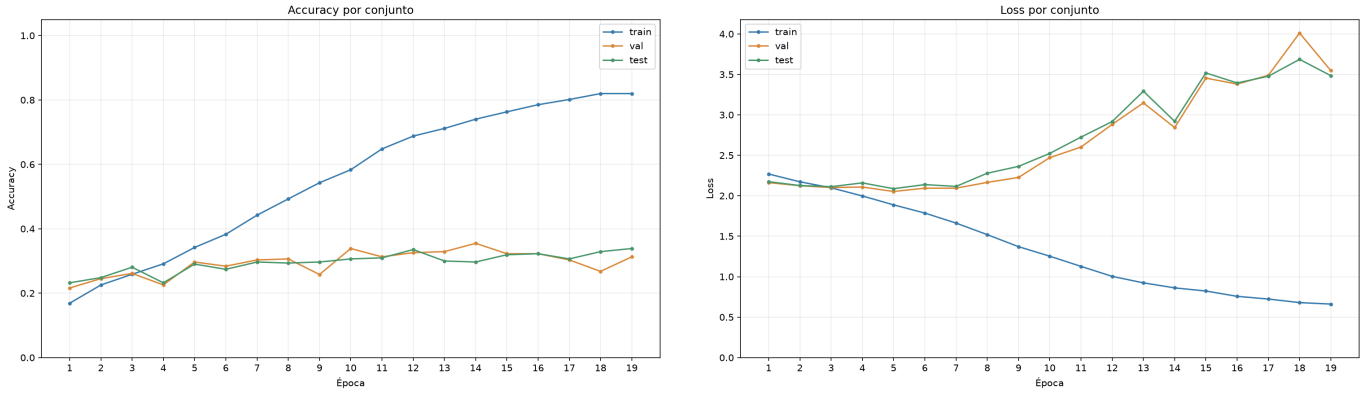

Fig. 10. Curvas conservadas de CNN v2 con L2. El ajuste de train sigue aumentando mientras validación y prueba presentan mejoras limitadas.

La Tabla IX presenta los resultados con los hiperparametros finales, la exactitud de train es 0.958 y la de prueba 0.297, esto representa una separación de 66.1 puntos porcentuales entre ambos conjuntos, el F1 macro de prueba de 0.274, lo que apenas supera el 0.273 de la CNN v1 original. Por tanto, con una red mucho mayor, dropout y L2 aumentaron considerablemente el rendimiento en el conjunto de train, pero no redujeron el comportamiento de overfitting, sino todo lo contrario.

TABLE IX
MÉTRICAS FINALES DE CNN V2 CON L2

Conjunto	Accuracy	Prec. macro	Recall macro	F1 macro
Train	0.958	0.959	0.959	0.959
Validación	0.355	0.350	0.349	0.336
Prueba	0.297	0.282	0.286	0.274

La Figura 10 muestra la separación entre entrenamiento y evaluación. La exactitud de train durante `fit` es menor que su evaluación final porque los pesos cambian dentro de la época y dropout permanece activo durante el aprendizaje. Evaluar el mejor modelo con dropout desactivado explica parte de esa diferencia de medición; no elimina la brecha frente a ejemplos reservados. Las curvas y las métricas finales señalan que el límite ya no se explica solo por falta de capacidad.

VIII. TRANSFER LEARNING CON EFFICIENTNET-B0

A. Conceptos utilizados

Transfer learning parte de características aprendidas en un conjunto grande para adaptarlas a otra clasificación, es decir, en lugar de estimar todos los filtros desde cero con pocas fotografías, se reutiliza una base preentrenada en ImageNet y se sustituye su clasificador. En el paper de Bilal et al. (2025) se compara entrenamiento desde cero, capas congeladas y ajuste fino en reconocimiento de herramientas, sus resultados motivan explorar esas estrategias cuando hay muchas diferencias visuales y pocos datos propios, sin garantizar tener la misma exactitud en carros.

Congelar la base significa impedir que sus pesos se actualicen y que unicamente el clasificador sea el que aprenda. *Fine tuning* permite después modificar una parte de la base con una tasa menor de aprendizaje, para adaptar características al dominio. Esta separación reduce el riesgo de alterar bruscamente los pesos preentrenados cuando la cabeza todavía está aprendiendo.

El *global average pooling* convierte cada mapa de características en su promedio espacial, así se obtiene un valor por canal sin conectar individualmente cada posición con la capa densa. En este modelo, la cabeza usa esos valores, dropout y una salida softmax de diez clases, comparado a la capa densa del modelo CNN v2, solo esta cabeza tiene 12,810 parámetros entrenables en la primera etapa.

La *reducción de la tasa de aprendizaje* disminuye la escala de las actualizaciones cuando una métrica deja de mejorar. Appasami y Savarimuthu (2025) integran ajustes de tasa y detención temprana (early stopping) en su proceso de optimización. En la configuración final del proyecto se distinguen dos funciones, la pérdida de validación controla cuándo reducir el learning rate y el F1 macro de validación determina los mejores pesos y la detención.

B. Implementación y configuración final

Se utiliza `EfficientNetB0` con pesos de ImageNet y sin su clasificador original, se utiliza global average pooling, dropout con valor de 0.5 y una capa densa softmax de diez salidas. Las entradas RGB se entregan como `float32` en $[0, 255]$, porque esta implementación incorpora su normalización. Los datos cargados tiene la forma de 320×320 con interpolación bilineal, antialias y relleno negro.

Primero se entrenó la cabeza con la base congelada, después se habilitan los bloques 6 y 7 del modelo de efficient net y la convolución superior, conservando congeladas todas las capas de BatchNormalization. Además, la base se invoca con `training=False` para mantener sus estadísticas de inferencia y desactivar su aleatoriedad interna, esto permite actualizar los pesos habilitados sin recalcular esas estadísticas. En la Tabla X se muestra la configuración final.

C. Resultados obtenidos

La base congelada ya obtiene F1 macro de prueba de 0.564, superior a las etapas de CNN v2 resumidas anteriormente.

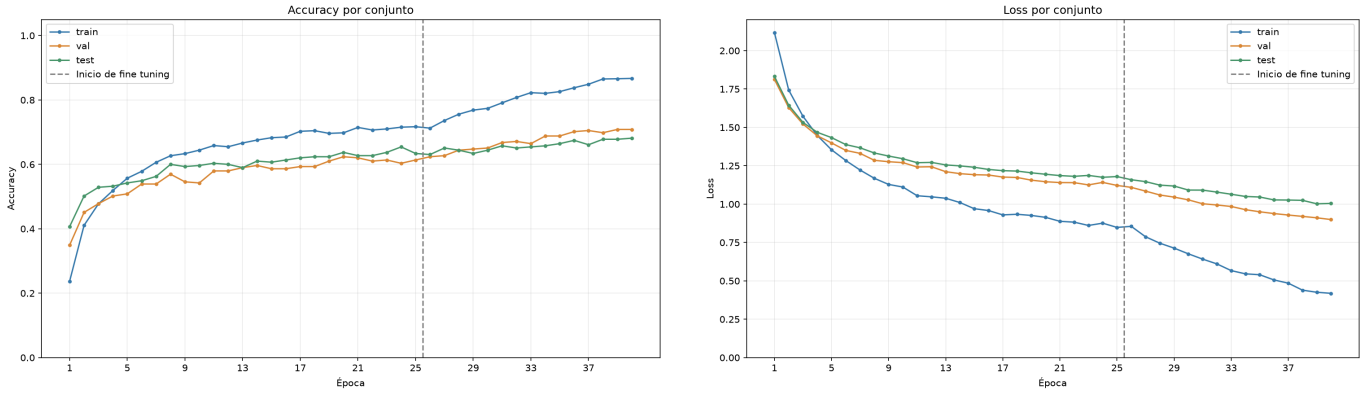

Fig. 11. Curvas de la configuración final de EfficientNet-B0. La línea vertical indica el inicio del ajuste fino después de entrenar la cabeza.

TABLE X
CONFIGURACIÓN FINAL DE EFFICIENTNET-B0

Elemento	Configuración
Entrada	RGB 320×320, intensidades en $[0, 255]$
Optimizador y pérdida	Adam y entropía cruzada con etiquetas enteras
Lote y semilla	32 imágenes; semilla 42
Cabeza	Dropout 0.5; máximo 30 épocas; tasa inicial 10^{-3}
Ajuste fino	Bloques 6 y 7 y `top_conv`; máximo 15 épocas; tasa inicial 10^{-5}
BatchNormalization	Congelada en ambas etapas
Reducción de tasa	Pérdida de validación; factor 0.5; paciencia 2; mínimo 10^{-7}
Selección de pesos	Mayor F1 macro de validación; paciencia 5 por etapa

Con la primera incorporación de ajuste fino y augmentación dinámica se eleva F1 de validación de 0.601 a 0.632, pero prueba apenas cambia de 0.564 a 0.567. Al regresar a augmentación fija, los valores de prueba se mantienen en 0.577 de exactitud y 0.567 de F1, esto muestra que añadir transformaciones o una segunda etapa no garantizó una mejora equivalente fuera de validación.

Posteriormente se amplió el ajuste fino para incluir el bloque 6 y se aumentó su máximo de 10 a 15 épocas, con este cambio se alcanza un F1 macro de prueba de 0.613. Después, al cargar imágenes de 320×320 en vez de reducirlas a 224×224, el F1 de prueba sube a 0.662, lo que indica que ahora se conserva más información de entrada, aun así, no demuestra por sí solo qué detalles utilizó la red. La Tabla XI presenta la secuencia completa, incluida la configuración final con limpieza.

Las curvas de la Figura 11 muestran mejoras de validación y prueba durante el ajuste fino, junto con una separación persistente respecto de train. Aunque la generalización mejora frente a las CNN propias, la red todavía aprovecha mejor los ejemplos de aprendizaje.

TABLE XI
EVOLUCIÓN HISTÓRICA DE EFFICIENTNET-B0

Etapa	Acc. prueba	F1 macro prueba
Base congelada	0.577	0.564
Primer ajuste fino y augmentación dinámica	0.577	0.567
Retorno a augmentación fija	0.577	0.567
Ajuste fino ampliado	0.619	0.613
Entrada a 320×320	0.665	0.662
Limpieza y configuración final	0.681	0.676

Nota. Valores redondeados de reportes históricos.

IX. REVISIÓN VISUAL DE LOS DATOS

A. *Exclusión de contenido poco pertinente*

Se realizó una revisión de imágenes que permitió identificar contenido ambiguo para reconocer la apariencia exterior de un carro. Se registraron 101 imagenes que se excluyeron, 20 fotografías de interiores de carros, 25 imágenes de LEGO, incluidas piezas, y 56 de anuncios. Las imagenes excluidas corresponden a las clases de BMW M3, Chevrolet Camaro y Ford Mustang, con 13, 37 y 51 archivos, respectivamente. El criterio utilizado para esta exclusión fue el contenido observado y su pertinencia para la tarea, no el valor del error que producía cada ejemplo.

Esta intervención se relaciona con el análisis exploratorio de Zhang et al. (2024), donde las muestras y los patrones visuales permiten formular hipótesis sobre problemas que los indicadores numéricos no describen por sí solos. Por ejemplo, aumentar con rotaciones una imagen de piezas de LEGO produce más variaciones de ese contenido, pero no una fotografía adicional de un vehículo real. Esa diferencia ayuda a explicar por qué conviene revisar datos incluso después de ajustar el modelo.

La exclusión de imagenes es parcial, pues no se revisaron exhaustivamente las diez clases ni se resolvieron todas las miniaturas, renders, detalles o etiquetas dudosas, tampoco se concluye que un interior sea inválido en cualquier tarea, solo que para este proyecto se prioriza reconocer el modelo por su apariencia exterior. Esta selección modifica la población evaluada y define la forma de reconocer las clases de los carros por el exterior.

B. Cambios en el conjunto de datos

Después de retirar las 101 imágenes, los datos crudos contienen $2{,}084 - 101 = 1{,}983$ registros. Al repetir el análisis y la limpieza automática se conservan 1,966 originales. La Tabla XII muestra la distribucion de imagenes para cada conjunto

TABLE XII
CONJUNTOS DE LA EJECUCIÓN FINAL TRAS LA REVISIÓN VISUAL

Conjunto	Originales	Entradas tras augmentación
Train	1,376	2,752
Validación	295	295
Prueba	295	295
Total	1,966	3,342

Para finalizar, después de esta limpieza de datos, se agregó `ReduceLROnPlateau`, que reduce la tasa a la mitad después de dos épocas sin mejorar la pérdida de validación, hasta un mínimo de 10^{-7}.

C. Resultados finales y errores que permanecen

La Tabla XIII recoge las métricas finales, en esta se muestra que la exactitud de prueba aumenta de 0.665 a 0.681 respecto de la ejecución inmediata anterior y F1 macro pasa de 0.662 a 0.676, un aumento de aproximadamente 1.6 y 1.4 puntos porcentuales, respectivamente. En el conjunto de validación, F1 macro sube de 0.692 a 0.706. La revisión visual forma parte de la mejor configuración final registrada, sin embargo no representó un gran salto en el desempeño final del modelo, puesto que el aumento previo de resolución elevó F1 de prueba de 0.613 a 0.662, unos 4.9 puntos.

TABLE XIII
MÉTRICAS FINALES DE EFFICIENTNET-B0 TRAS LA REVISIÓN

Conjunto	Accuracy	Prec. macro	Recall macro	F1 macro
Train	0.969	0.969	0.970	0.970
Validación	0.708	0.715	0.706	0.706
Prueba	0.681	0.693	0.674	0.676

La matriz de la Figura 12 permite verificar $201/295 \approx 0.681$ de exactitud. Audi R8 obtiene 32 aciertos de 34 y Nissan GT-R, 28 de 33, sin embargo, persisten dificultades en clases visualmente cercanas, como en BMW M3 que tiene 16 aciertos de 33 y diez de sus imágenes se asignan a BMW M5. Chevrolet Camaro y Ford Mustang tienen 16 y 17 aciertos de 28, respectivamente, con seis errores hacia la otra clase.

La exactitud final de train de 0.969 permanece 28.8 puntos por encima del conjunto de test. Esa brecha y el F1 macro de 0.676 muestran que la transferencia y la revisión de datos ayudan, pero todavía existe un problema de falta de generalización y overfitting.

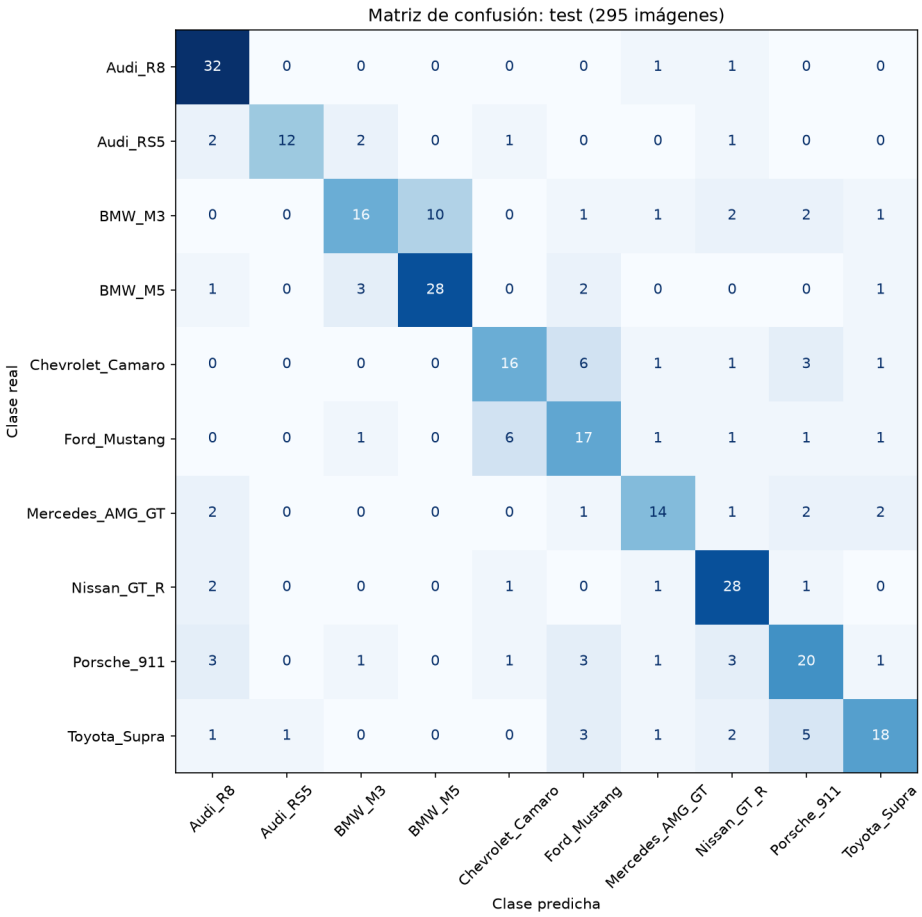

Fig. 12. Matriz de confusión de EfficientNet-B0 en la prueba final de 295 imágenes. La diagonal suma 201 aciertos.

X. Comparación general

La CNN v1 estableció una referencia pequeña y mostró que la reducción espacial fuerte y el aprendizaje desde cero eran insuficientes para este conjunto. Con la CNN versión 2 se buscó conservar más detalle y controlar el sobreajuste mediante dropout, augmentación y L2. Sin embargo, aumentar de 115,242 a más de seis millones de parámetros hizo mucho más fácil ajustar train y no produjo una ganancia equivalente en prueba. Entre las etapas resumidas, la augmentación alcanzó F1 de prueba de 0.325, mientras que la configuración posterior elegida por validación obtuvo 0.274.

EfficientNet-B0 aporta una representación aprendida previamente y alcanza 0.564 de F1 de prueba incluso con su base congelada. El ajuste fino ampliado, la mayor resolución y la configuración final llevan ese indicador a 0.676. La diferencia respecto de CNN v1 es de 40.3 puntos en F1 macro y 37.1 puntos en exactitud.

XI. Conclusiones

El proyecto evolucionó desde una CNN compacta con desempeño limitado hasta una solución preentrenada que reconoce mejor las diez clases. La CNN v1 original obtuvo exactitud de prueba de 0.310 y F1 macro de 0.273. CNN v2 permitió explorar regularización, resolución y búsqueda de hiperparámetros, pero mostró que ajustar casi por completo train no equivale a reconocer nuevos ejemplos, porque su reporte posterior con L2 alcanzó F1 de prueba de 0.274, pese a una ventaja en validación dentro de su búsqueda, la etapa previa con augmentación obtuvo 0.325 y constituye una mejora observada dentro de esa secuencia.

La transferencia de aprendizaje produjo el cambio más claro entre las opciones analizadas. EfficientNet-B0 partió de F1 macro de prueba de 0.564 con la base congelada y alcanzó 0.676 con la configuración final, además de exactitud de 0.681. Ajustar parcialmente los pesos y conservar mayor resolución resultó más útil en estas ejecuciones que seguir ampliando la CNN propia.

La revisión de 101 imágenes de interiores, LEGO y anuncios aportó una mejora final relevante para definir qué debía aprender el sistema. Con esto se alcanzó una mejora en F1 del conjunto de test de 0.662 a 0.676, pero también cambia la partición y el control del learning rate.

Persisten sobreajuste, errores entre BMW M3 y M5 y entre Camaro y Mustang, variación de calidad y etiquetas ambiguas. Los principales problemas enfrentados fueron distinguir capacidad de generalización, sostener comparaciones

al cambiar el procesamiento y el dataset, y asumir el costo de una búsqueda cuyos avances en validación no siempre llegaban a prueba. El siguiente paso debe combinar mejores datos con una evaluación estable y repetida, para comprobar si los avances se mantienen fuera de las imágenes utilizadas durante el desarrollo.

Referencias

Appasami, G., & Savarimuthu, N. (2025). A novel lightweight CNN design for MRI brain tumor image classification with performance-driven optimization. *Discover Computing, 28*, Article 206. https://doi.org/10.1007/s10791-025-09603-4

Baressi Šegota, S., Mrzljak, V., Lorencin, I., & Anđelić, N. (2025). Data augmentation-driven improvements in malignant lymphoma image classification. *Computers, 14*(7), Article 252. https://doi.org/10.3390/computers14070252

Bilal, M., Podishetti, R., Grossmann, D., & Bregulla, M. (2025). Benchmarking CNN architectures for tool classification: Evaluating CNN performance on a unique dataset generated by novel image acquisition system. *IEEE Access, 13*, 96400–96422. https://doi.org/10.1109/ACCESS.2025.3574785

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org/

Keras. (s.f.-a). *EarlyStopping*. Recuperado el 26 de octubre de 2026, de https://keras.io/api/callbacks/early_stopping/

Keras. (s.f.-b). *MaxPooling2D layer*. Recuperado el 26 de octubre de 2026, de https://keras.io/api/layers/pooling_layers/max_pooling2d/

Khan, M. S., Siivola, E., & Raitoharju, J. (2026). On class imbalance in machine learning-based taxa identification: A comparative analysis of mitigation strategies. *Machine Learning with Applications, 24*, Article 100931. https://doi.org/10.1016/j.mlwa.2026.100931

Kingma, D. P., & Ba, J. (2015). *Adam: A method for stochastic optimization*. International Conference on Learning Representations. https://arxiv.org/abs/1412.6980

lightangle. (s.f.). *Car model image classification* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/lightangle/car-model-image-classification

Sahaya Sakila, V., Sujeetha, R., Ramanathan, S. V., Adhithya Raj, M., & Philip Regin, B. (2025). A benchmark comparison of simple CNN, ResNet-50, and EfficientNet for image classification. *Archives for Technical Sciences, 33*(2), 535–547. https://doi.org/10.70102/afts.2025.1833.535

van den Hoogen, J., Hudson, D., Bloemheuvel, S., & Atzmueller, M. (2024). Hyperparameter analysis of wide-kernel CNN architectures in industrial fault detection: An exploratory study. *International Journal of Data Science and Analytics, 18*, 423–444. https://doi.org/10.1007/s41060-023-00440-6

Wojciuk, M., Swiderska-Chadaj, Z., Siwek, K., & Gertych, A. (2024). Improving classification accuracy of fine-tuned CNN models: Impact of hyperparameter optimization. *Heliyon, 10*, Article e26586. https://doi.org/10.1016/j.heliyon.2024.e26586

Zhang, Y., Allen, T. T., & Rodriguez Buno, R. (2024). Exploratory image data analysis for quality improvement hypothesis generation. *Quality Engineering, 36*(4), 693–712. https://doi.org/10.1080/08982112.2023.2285305